

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

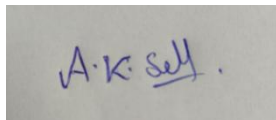
Reverse Engineering Task

Code of Conduct:

I A K Selva Prabhu(192421224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink that reads "A.K. Selva".

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

Answer:

1. Understanding of the Existing System

The existing E-Learning Management System manages student registration, course management, content delivery, assignment evaluation, and result generation. The main problem is that the user interface code and business rules are tightly coupled. Therefore, making changes to one part of the system may affect other modules.

2. Problems in the Existing System

- User interface and business logic are tightly coupled.
- Large modules have multiple responsibilities.
- Changes are difficult to implement.
- Code reuse is limited.
- Testing individual components is difficult.
- Maintenance becomes complex as the system grows.
- Poor separation of concerns reduces scalability.

3. Reverse Engineered Classes

Student

- Attributes: studentId, name, email
- Methods: register(), login(), viewCourse()

Course

- Attributes: courseId, courseName, description
- Methods: addCourse(), updateCourse(), viewCourse()

Content

- Attributes: contentId, title, type
- Methods: uploadContent(), viewContent()

Assignment

- Attributes: assignmentId, title, deadline
- Methods: submitAssignment(), evaluateAssignment()

Result

- Attributes: resultId, marks, grade
- Methods: calculateGrade(), generateResult()

4. Responsibilities and Relationships

- A **Student** can enroll in multiple **Courses**.
- A **Course** contains multiple **Content** objects.
- A **Course** can contain multiple **Assignments**.
- An **Assignment** is submitted and evaluated for a Student.
- The evaluation produces a **Result**.

5. Redesigned Layered Architecture

Presentation Layer:

Provides screens for registration, course selection, content viewing, assignment submission, and result viewing.

Service Layer:

Handles student registration, course enrollment, assignment evaluation, and result generation.

Domain Layer:

Contains Student, Course, Content, Assignment, and Result classes.

Data Access Layer:

Handles storing and retrieving student, course, assignment, and result information.

6. Redesign

The system should separate the user interface from business logic by using layered architecture. Each class should have a clear responsibility. Communication between layers should be performed through defined interfaces rather than directly accessing internal components.

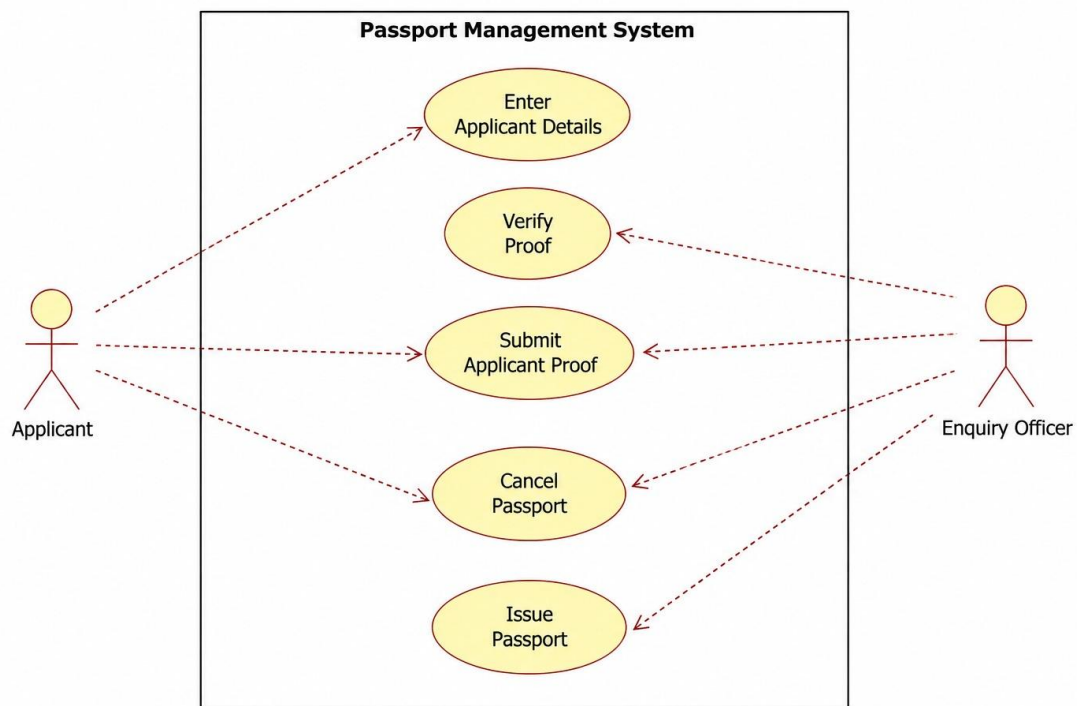
7. Benefits

- **High Cohesion:** Each class performs a specific responsibility.
- **Loose Coupling:** Changes in one layer have less effect on other layers.
- **Maintainability:** Individual modules can be modified easily.

- **Scalability:** New learning features can be added without redesigning the complete system.
- **Reusability:** Common classes and services can be reused.
- **Testability:** Individual layers and classes can be tested separately.

Conclusion

The redesigned object-oriented and layered structure improves the overall quality of the E-Learning Management System by providing better cohesion, lower coupling, improved maintainability, and greater scalability.



2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and

explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

Answer:

1. Understanding of the Existing System

The existing customer support application handles requests through email, chatbot, telephone, and social media. It uses multiple conditional statements to determine how each request should be processed. Adding a new support channel requires changes to several existing modules.

2. Problems in the Existing Implementation

- Excessive conditional statements.
- Strong dependency between modules.
- Adding a new support channel requires modifying existing code.
- Difficult to maintain.
- Poor flexibility and extensibility.
- Code duplication may occur.
- Testing becomes difficult when many conditions are present.

3. Reverse Engineered Components

SupportRequest

- Attributes: requestId, customerId, message
- Methods: createRequest(), getRequestDetails()

SupportChannel

- Methods: sendRequest(), receiveRequest()

EmailSupport

- Methods: sendRequest()

ChatbotSupport

- Methods: sendRequest()

TelephoneSupport

- Methods: sendRequest()

SocialMediaSupport

- Methods: sendRequest()

4. Suitable Behavioral Design Pattern

The **Strategy Pattern** can be used to handle different support channels.

A common SupportChannel interface can define the operation handleRequest().

Different implementations can be created:

- EmailSupport
- ChatbotSupport
- TelephoneSupport
- SocialMediaSupport

The system can select the required strategy at runtime based on the support channel.

5. Redesigned Structure

SupportChannel Interface

↓

EmailSupport

ChatbotSupport

TelephoneSupport

SocialMediaSupport

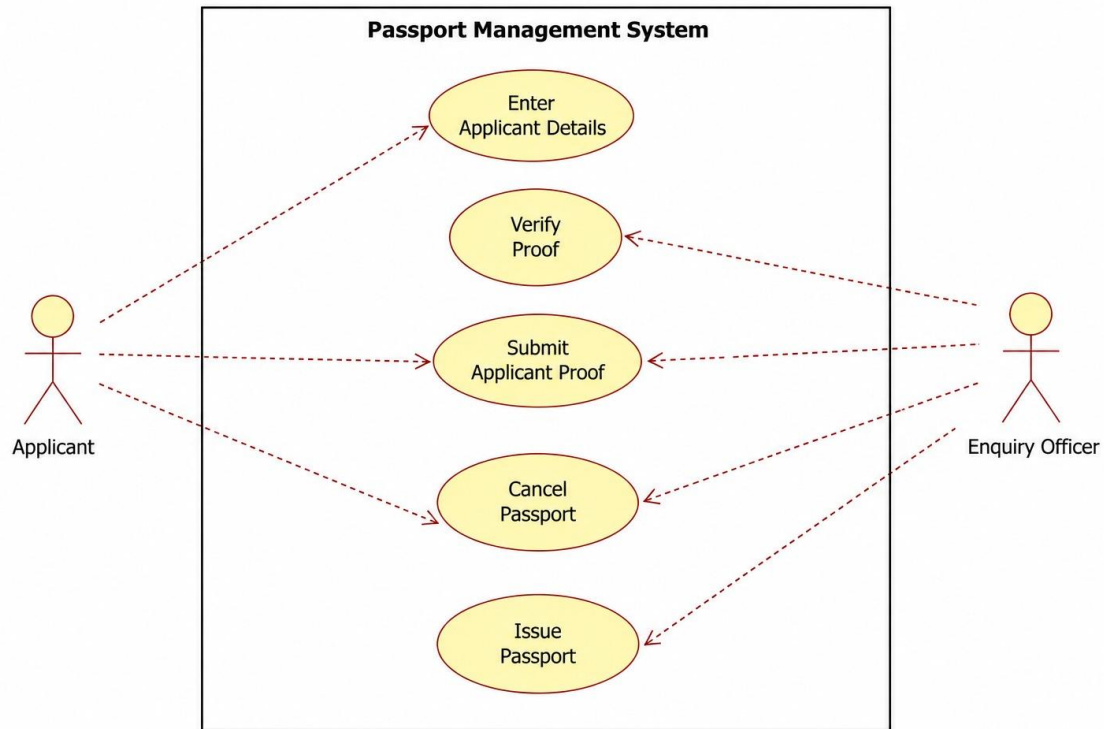
The main support service depends on the SupportChannel abstraction instead of directly depending on individual channels.

6. Benefits

- **Flexibility:** Different support channels can be selected easily.
- **Extensibility:** New channels can be added without changing existing support logic.
- **Loose Coupling:** The main service depends on an abstraction.
- **Maintainability:** Each channel has its own implementation.
- **Reusability:** Support channel components can be reused.
- **Testability:** Each support channel can be tested independently.

Conclusion

Using a behavioral design pattern such as the Strategy Pattern removes excessive conditional statements and provides a flexible structure. New channels can be introduced with minimal changes to the existing system.



3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes, service classes, and data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

Answer:

1. Understanding of the Existing System

The Legacy Banking Transaction System manages account management, transaction validation, fund transfer, loan processing, and transaction reporting. These functions are contained in a few tightly coupled classes. Database operations are also directly embedded inside business methods.

2. Problems in the Existing System

- Business logic and database operations are mixed.
- Classes have multiple responsibilities.
- High coupling between components.
- Difficult to test business logic independently.
- Database changes may affect business logic.
- Code maintenance is difficult.
- Reusability is reduced.

3. Reverse Engineered Domain Classes

Customer

- Attributes: customerId, name, contact
- Methods: viewAccount(), updateDetails()

Account

- Attributes: accountId, balance, accountType
- Methods: deposit(), withdraw(), checkBalance()

Transaction

- Attributes: transactionId, amount, transactionType
- Methods: createTransaction(), validateTransaction()

Loan

- Attributes: loanId, amount, interestRate
- Methods: processLoan(), calculateInterest()

TransactionReport

- Attributes: reportId, date
- Methods: generateReport()

4. Service Classes

AccountService

- Manages account operations.

TransactionService

- Handles transaction validation and fund transfers.

LoanService

- Handles loan processing and interest calculation.

ReportService

- Generates transaction reports.

5. Data Access Components

AccountDAO

- Stores and retrieves account information.

TransactionDAO

- Stores and retrieves transaction objects.

LoanDAO

- Stores and retrieves loan objects.

6. Redesigned Layered Architecture

Presentation Layer

↓

Service Layer

↓

Domain Layer

↓

Data Access Layer

↓

OODBMS

The Presentation Layer receives user requests. The Service Layer performs business operations using Domain objects. The Data Access Layer handles persistence and communicates with the OODBMS.

7. Suitable Design Patterns

DAO Pattern:

Separates database operations from business logic.

Repository Pattern:

Provides a common interface for storing and retrieving persistent domain objects.

Strategy Pattern:

Can be used for different loan-interest calculation policies.

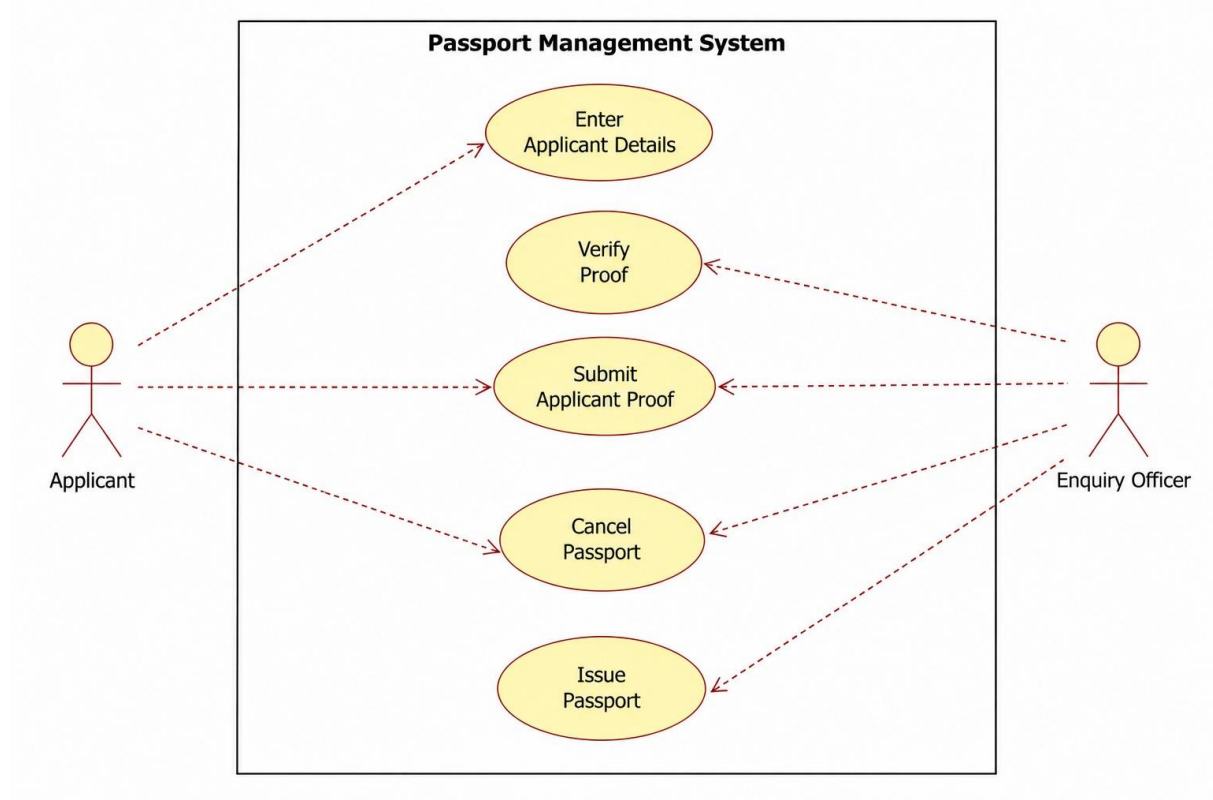
8. Benefits

- **Separation of Concerns:** Each layer has a clear responsibility.
- **Testability:** Business logic can be tested independently.
- **Maintainability:** Database changes do not directly affect business logic.

- **Reusability:** Services and data access components can be reused.
- **Loose Coupling:** Components communicate through defined interfaces.
- **Scalability:** New banking services can be added more easily.

Conclusion

The redesigned banking system separates business operations from database operations. The layered architecture and design patterns improve separation of concerns, testability, reusability, and maintainability.



4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO, Repository, or Data Access patterns**, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

Answer:

1. Understanding of the Existing System

The Legacy Hotel Reservation System directly performs OODBMS operations from reservation screens. Database queries for rooms, guests, bookings, and payments are repeated across different modules.

2. Problems in the Existing Architecture

- Reservation screens directly communicate with the database.
- High coupling between UI and OODBMS.
- Repeated database queries.
- Code duplication.
- Difficult database maintenance.
- Poor reusability.
- Difficult testing of individual components.

3. Reverse Engineered Classes

Guest

- Attributes: guestId, name, contact
- Methods: registerGuest(), updateGuest()

Room

- Attributes: roomId, roomType, price, status
- Methods: checkAvailability(), updateRoom()

Booking

- Attributes: bookingId, checkInDate, checkOutDate
- Methods: createBooking(), cancelBooking()

Payment

- Attributes: paymentId, amount, paymentStatus
- Methods: makePayment(), verifyPayment()

4. Data Access Components

GuestDAO

- saveGuest()
- findGuest()
- updateGuest()

RoomDAO

- saveRoom()
- findAvailableRoom()
- updateRoom()

BookingDAO

- saveBooking()
- findBooking()
- cancelBooking()

PaymentDAO

- savePayment()
- findPayment()

5. Repository Pattern

A Repository can provide a common interface between business components and persistent hotel objects.

For example:

RoomRepository

- save()
- findById()
- findAvailableRooms()
- update()

BookingRepository

- save()
- findById()
- cancel()

6. Redesigned Architecture

Presentation Layer

↓

Service/Business Layer

↓

Domain Layer

↓

DAO/Repository Layer

↓

OODBMS

The Presentation Layer does not directly access the OODBMS. All database operations are handled through DAO or Repository components.

7. Benefits

- **Modularity:** Each component has a specific responsibility.
- **Reusability:** Common database operations can be reused.
- **Reduced Duplication:** Repeated queries are centralized.
- **Loose Coupling:** UI is separated from persistence.
- **Maintainability:** Database operations can be modified independently.
- **Better OODBMS Integration:** Persistent hotel objects are managed through dedicated data access components.

Conclusion

Introducing DAO and Repository patterns removes direct database dependency from reservation screens. The redesigned architecture improves modularity, reusability, maintainability, and OODBMS integration.

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

Answer:

1. Understanding of the Existing System

The Legacy Transportation Management System manages buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated in several parts of the application. This makes it difficult to introduce new transportation services.

2. Problems in the Existing System

- Object creation logic is duplicated.
- High dependency on concrete classes.
- Adding a new transportation type requires changes in multiple modules.
- Code reuse is low.
- Maintenance becomes difficult.
- System extensibility is limited.

3. Common Abstraction

A common **Transportation** abstraction can be created.

Transportation

- Attributes: vehicleId, capacity
- Methods: start(), stop(), getDetails()

The following classes can inherit from it:

- Bus
- Train
- Taxi
- RentalVehicle

4. Object Relationships

Transportation

↓

Bus

Train

Taxi

RentalVehicle

The common Transportation abstraction contains common operations, while each subclass provides its specific implementation.

5. Factory Method Pattern

A **TransportationFactory** can be introduced to create transportation objects.

The application requests a transportation type from the Factory rather than directly creating objects.

For example:

- "Bus" → Bus object
- "Train" → Train object
- "Taxi" → Taxi object
- "RentalVehicle" → RentalVehicle object

6. Redesigned Structure

Client

↓

TransportationFactory

↓

Transportation Interface/Class



Bus / Train / Taxi / RentalVehicle

The client does not need to know the exact object creation process.

7. Benefits

- **Extensibility:** New transportation types can be added easily.
- **Scalability:** The system can support additional transportation services.
- **Code Reuse:** Common functionality is maintained in the parent abstraction.
- **Loose Coupling:** Client code does not depend directly on concrete classes.
- **Maintainability:** Object creation logic is centralized.
- **Flexibility:** Different transportation objects can be created dynamically.

Conclusion

The Factory Method Pattern provides a clean way to manage transportation object creation. It removes duplicated creation logic and makes the system easier to extend, scale, reuse, and maintain.

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

Answer:

1. Understanding of the Existing System

The Legacy Healthcare Monitoring System collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete monitoring devices and storage components.

2. SOLID Violations

Single Responsibility Principle (SRP):

Some classes perform multiple unrelated tasks such as collecting sensor data, processing readings, storing records, and generating reports.

Open/Closed Principle (OCP):

Adding a new monitoring device or new processing method may require modifying existing classes.

Dependency Inversion Principle (DIP):

High-level classes directly depend on concrete monitoring devices and storage implementations instead of abstractions.

3. Reverse Engineered Classes**Patient**

- Attributes: patientId, name, age
- Methods: updatePatientDetails(), viewRecords()

Sensor

- Methods: readData()

HeartRateSensor

- Methods: readData()

TemperatureSensor

- Methods: readData()

AlertService

- Methods: generateAlert()

MedicalRecord

- Attributes: recordId, reading, date
- Methods: saveRecord(), viewRecord()

ReportService

- Methods: generateReport()

4. Redesigned Abstractions

Create a common **Sensor Interface**:

Sensor

- readData()

Different sensors implement the interface:

- HeartRateSensor
- TemperatureSensor
- OtherSensor

The monitoring service depends on the Sensor interface instead of concrete sensor classes.

5. Storage Abstraction

Create a **MedicalRecordRepository Interface** with methods such as:

- saveRecord()
- findRecord()
- updateRecord()

Different storage implementations can implement this interface.

The high-level monitoring service depends on the repository abstraction rather than a specific database implementation.

6. Observer Pattern

The **Observer Pattern** can be used for monitoring alerts.

When abnormal sensor readings are detected:

Sensor/Monitoring Service → Alert Service → Registered Users/Systems

This allows alerts to be generated without tightly coupling the monitoring component to every notification component.

7. Redesigned Architecture

Presentation Layer

↓

Monitoring Service

↓

Sensor Interface → Different Sensors

↓

Medical Record Repository

↓

Storage/OODBMS

Alert services can receive notifications when abnormal readings are detected.

8. Benefits

- **High Cohesion:** Each class performs a focused responsibility.
- **Loose Coupling:** Components depend on interfaces rather than concrete classes.
- **Flexibility:** New sensors can be added without modifying the main monitoring service.
- **Testability:** Sensors and repositories can be replaced with test implementations.
- **Maintainability:** Individual components can be modified independently.
- **Extensibility:** New alert mechanisms and storage systems can be added easily.

Conclusion

Applying SRP, OCP, and DIP along with suitable abstractions and design patterns creates a more flexible and maintainable healthcare monitoring system. The redesigned architecture supports new sensors, storage technologies, and monitoring features while reducing dependencies between components.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation